

SAP ABAP AMDP / SQLScript / Code-to-Data 課程

REST 課程 ([src/ABAP_Training_REST/](#) · rs01~rs11) 之後的下一階段候選之一。課綱草案 (2026-07-19) · **am01 ~ am09** 全部出題完成 (**2026-07-19**) · **am09** 期末整合待使用者驗收。

課程定位

- **對象**：完成 OOP / REST 課程的學員 (會 Class/Interface 、 Open SQL 、 BAPI 呼叫)
- **技術範圍**：AMDP (ABAP Managed Database Procedure) + SQLScript 語法本身 + Code-to-Data (把運算邏輯下推到資料庫層) 的觀念，範圍限定在 AMDP 這條線，**不含** RAP/CDS 那條現代化路線的其餘主題 (Behavior Definition 、 Service Binding 等留待另一門課)
- **前置知識缺口**：前面所有課程 (基礎課/OOP/REST) 用的都是 Open SQL——ABAP 應用層迴圈 + SQL 讀寫；AMDP 是完全不同的典範 (邏輯本身跑在資料庫引擎裡)，SQLScript 又是一個新語法，所以這門課要花比 REST 課更多篇幅在「語法本身」，不能只教「怎麼包一個 AMDP Method」
- **系統確認**：目前連線系統是 On-Premise S/4HANA 1909 ([SAP BASIS 754](#))，資料庫是 **SAP HANA 2.0 SPS04** ([DBSystem=HDB](#))——AMDP 硬性要求 HANA 資料庫，這套系統符合；ADT Discovery 已確認有 [AMDP Debugger](#)、[Data Preview for AMDP](#)、[ABAP Database Procedure Proxies](#) 等工具鏈，可以在這套系統直接開發與偵錯，不需要額外環境
- **結業標準 (草案)**：能判斷一段邏輯該留在 ABAP 應用層還是下推到 AMDP、能獨立寫一個中等複雜度的 SQLScript 程序 (含變數、流程控制、多表 JOIN/聚合)、知道怎麼從 ABAP 呼叫並處理例外、能把 AMDP 包成 CDS Table Function 供 CDS View 使用、**能寫一個同時 EXPORTING 兩個 (以上) Internal Table 的 AMDP Method**——這是 AMDP 跟一般 ABAP Method ([RETURNING](#) 只能單一值) 本質不同的地方，也是 SQLScript 的重要特性，課程至少要有一題明確示範

教材慣例 (比照 OOP/REST 課程)

- 每題三件套：題目 [amNN_主題.md](#) + PDF 講義 ([node tools/md2pdf.js src/ABAP_Training_AMDP](#)) + 答案快照
- 每題 md 開頭 ([## 學習目標](#) 之前) 要有 [## Lecture](#) 完整背景知識講解，延續 REST 課程 rs07 起養成的慣例
- 答案物件命名：AMDP 本質上還是 ABAP 類別 + 方法 ([BY DATABASE PROCEDURE](#) 或 [BY DATABASE FUNCTION](#))，沿用 [ZCL_AMnn_*](#)；CDS Table Function 的 DDL Source 命名為 [ZTF_AMnn_*](#) (am07 起實測確認)，套件 [\\$TMP](#)
- 資料模型：優先沿用 SCARR/SFLIGHT/SBOOK 航班模型 (跟 OOP/REST 一致)，除非某題需要更大量測試資料才能看出 Code-to-Data 的效能差異，才考慮換模型或造測試資料

課綱 (草案，待確認與逐題出題)

#	主題	內容重點	銜接前面課程	狀態
am01	為什麼要 Code-to-Data + AMDP 架構總覽	Open SQL「搬資料回應層層算」vs AMDP「邏輯下推到資料庫算」的本質差異；AMDP = ABAP Method 包一段 SQLScript； IF_AMDP_MARKER_HDB 、 BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT 語法骨架；實測發現 AMDP 簽章限制 (所有參數須 VALUE() 、 DEFAULT 只能常數不能 sy-mandt) 與最重要的一課： AMDP 不會像 Open SQL 自動做 Client 過濾 ， SCARR 在這套系統多 Client 灌了同一批資料，不加 WHERE mandt = :iv_mandt 會撈出重複資料	對照 REST rs01 的破題方式；呼應 RAP/CDS 討論裡「這套系統是 HANA」的前提	已驗收
am02	第一個 AMDP Method：	AMDP Method 參數限制 (修正 ：elementary 純量型別跟 Table Type 都可以用，am01/am02 的 iv_mandt/iv_carrid 就是 elementary 純量參數；真正不	承 OOP op02 方法參數的對照；ALV 呈現	已驗收

#	主題	內容重點	銜接前面課程	狀態
	Signature 規則與呼叫方式	能用的是「非 Table 包裝的 Structure」；只有 IMPORTING/EXPORTING，沒有 RETURNING/CHANGING；從一般 ABAP 呼叫 AMDP Method 跟呼叫一般 Method 語法上完全一樣（呼叫端無感）；同時 EXPORTING 兩個 Internal Table 的範例（ ZCL_AM02_FLIGHT_STATS ：同一次 SQLScript 運算，一次吐出 SFLIGHT 明細 + 依 connid 分組的彙總統計兩張表），對照一般 ABAP Method 只能 RETURNING 單一值/表格的限制；呼叫端 ZR_AM02_DEMO 用 c1_salv_table 把這兩張回傳的 Internal Table 各自呈現成一個 ALV （ALV 需要 GUI，已用 WRITE 版本自行驗證過資料邏輯正確，畫面效果經使用者於 SAP GUI 執行確認無誤）	承 OOP op11 c1_salv_table	
am03	SQLScript 基本語法：變數、流程控制	ZCL_AM03_PRICE_TIER ：DECLARE 純量變數 + DECLARE CURSOR、FOR ... AS <cursor_name> DO ... END FOR 迴圈、IF/ELSEIF/ELSE 分支，逐筆分類票價高低並累計三個等級筆數，對照另一段用宣告式 CASE WHEN 一次 SELECT 做同樣分類的寫法；實測發現 FOR <var> AS SELECT ... （直接內嵌查詢）語法不合法，必須先 DECLARE CURSOR 再用游標名稱，已在講義記錄	—	已驗收
am04	SQLScript 集合處理：多表 JOIN / 聚合 / CTE	ZCL_AM04_ROUTE_LOAD ：WITH ... AS (...) CTE 先依航線彙總 SFLIGHT，再 JOIN SCARR 補上公司名稱算出每條航線載客率；實測發現兩個坑：AMDP 簽章 USING 子句多個物件要空白分隔、不是逗號；SQLScript 的 JOIN ON 條件必須明寫 MANDT（跟 Open SQL JOIN ON 條件不能寫 MANDT 正好相反，兩者都指向「Client 處理自動化在哪一層」這個核心對照）	對照 REST rs05 的 WHERE 條件組合手法；呼應 .claude/rules/sap-adt-mcp.md 第 10 節 Open SQL JOIN 限制	已驗收
am05	錯誤處理與例外	ZCL_AM05_FLIGHT_VALIDATOR ：DECLARE ... CONDITION FOR SQL_ERROR_CODE + SIGNAL 主動拋錯，ABAP 端 RAISING cx_amdp_error + TRY...CATCH 攔截；實測發現 SIGNAL 的條件名稱必須先 DECLARE ... CONDITION 宣告，不能直接用（原本以為 SQLSCRIPT_ERROR 是內建可用的名稱）；get_text() 拿到的是技術性訊息（含程序名/行號），自訂文字埋在最後面，不適合直接給使用者看；AMDP 沒辦法呼叫回 ABAP（單向限制）	承 OOP op09 例外類別設計	已驗收
am06	AMDP 除錯與資料預覽	ZR_AM06_DEMO ：重用 am04 已驗證的 AMDP 方法，改用經典 ALV （ REUSE_ALV_GRID_DISPLAY Function Module，手動組 IT_FIELDCAT ）呈現結果，對照 op11/am02 教過的 c1_salv_table （Functional ALV，靠型別反射自動產生欄位目錄）兩種世代的差異；Eclipse ADT 內建 AMDP Debugger 操作步驟（本題唯一需要使用者在 Eclipse 手動操作，Claude 端無法自動驗證）、Data Preview 快速查資料現況	—	已驗收
am07	CDS Table Function：AMDP 的	ZTF_AM07_ROUTE_STATS + ZCL_AM07_ROUTE_STATS ：把 am04 的航線載客率邏輯包成 CDS Table Function， ZR_AM07_DEMO 純 Open SQL SELECT FROM 查詢（完全不	呼應 RAP/CDS 討論的技術背景	已驗收

#	主題	內容重點	銜接前面課程	狀態
	另一個身分	呼叫 AMDP Method) ; 實測三個坑 : returns 結構須有 abap.clnt 型別欄位放第一位 (底層 Client 相關表會被要求) 、 FOR TABLE FUNCTION 只能寫在 CLASS DEFINITION 不能寫在 IMPLEMENTATION 、實作方法要用 BY DATABASE FUNCTION 不是 BY DATABASE PROCEDURE ; 關鍵發現 : 包成 Table Function 後透過 Open SQL 查詢會自動做 Client 過濾 (26 筆跟 am04 一致) , 是 am01 「AMDP 不自動處理 Client」教訓的重要例外		
am08	Code-to-Data 實戰改寫	ZCL_AM08_REVENUE_CLASSIC (改寫前 : SELECT+LOOP 逐筆算營收 , 沿用 op12 航班營收模型) vs ZCL_AM08_FLIGHT_REVENUE (改寫後 : AMDP 一次 SELECT 內建 CAST 計算) , ZR_AM08_DEMO 用 GET RUN TIME FIELD 比對兩者筆數/總營收/耗時 ; 關鍵發現 : AMDP 第一次呼叫因執行計畫編譯耗時 208ms (比 ABAP 版 33ms 慢) , 後續呼叫降到 2~13ms (比 ABAP 版更快) ——Code-to-Data 不是無條件更快 , 要考慮呼叫頻率與資料量	對照 OOP op12 報表重構的定位	已驗收
am09	綜合實作 : 分析型報表用 AMDP + CDS Table Function 呈現	ZTF_AM09_CARRIER_STATS + ZCL_AM09_CARRIER_STATS : 兩層 CTE (先依航線彙總 , 再捲成依公司彙總) , 整合 am04 (JOIN/聚合) + am07 (Table Function) + am08 (營收計算) ; ZR_AM09_CAPSTONE 純 Open SQL + cl_salv_table , 完全沒有業務邏輯 ; 驗證方式 : 拿 am04/am08 已驗證過的細顆粒度數字手動加總 , 跟這題公司層級彙總結果交叉核對 , LH 的 route_cnt/total_flights/seats_occ/seats_max/total_revenue 全部吻合	對照 REST rs09 期末整合的收斂角色	待使用者確認 (已用 programrun API 驗證資料邏輯 , 與 am04/am08 交叉核對一致 ; ALV 畫面待 SAP GUI 確認)

不碰的範圍 (明確排除)

RAP (Behavior Definition/Service Binding 等) 、CDS View 的完整 Annotation 體系、HANA PlanViz 效能調校細節、SQLScript 進階特性 (Table UDF 、Scalar UDF 、Graph 相關) ——這些留給有需要時另開課或另一階段。

出題工作流程 (比照 OOP/REST 課程)

SAP 建物件 (**sap_create_object** ; CDS Table Function 等 MCP 不支援的物件類型見 [.claude/rules/sap-adt-mcp.md](#) 的 ADT API workaround , 需要時另外補充 AMDP/DDLS 專屬的建立方式) → **sap_set_source** 寫入 → 語法檢查 + 啟用 → 使用者用 ADT 或 SE38 實測 (AMDP 無 SICF 需求 , 不需要瀏覽器測試) → 使用者驗收 → 快照 → 題目 md → **node tools/md2pdf.js src/ABAP_Training_AMDP** 產 PDF → 更新本 README 狀態 → commit 。每批 2-3 題、使用者驗收後再繼續。